

Interfaz Gráfica Web para la Visualización 3D de la Sabana de Bogotá Mediante Estilos de Línea: Un Análisis Técnico

1. Introducción

1.1. Objetivo y Alcance del Proyecto

Este informe tiene como objetivo proporcionar una evaluación técnica experta y una guía estratégica para el desarrollo de una Interfaz Gráfica de Usuario (GUI) basada en la web, dedicada a la visualización tridimensional (3D) de un modelo que representa la Sabana de Bogotá. El enfoque central reside en el empleo de diversas técnicas de renderizado basadas en estilos de línea para articular la morfología del terreno.

El alcance del proyecto abarca la adquisición y preparación de datos, la selección de una pila tecnológica web 3D apropiada, la exploración detallada de metodologías de visualización basadas en líneas (como mallas de alambre o *wireframes*, curvas de nivel y efectos avanzados basados en *shaders*), consideraciones de diseño de UI/UX para una interacción intuitiva, y estrategias para asegurar un rendimiento óptimo con conjuntos de datos geoespaciales potencialmente masivos.

1.2. Definiendo "La Sabana": Contexto Geográfico

Con base en la información investigada, "la Sabana" se refiere a la Sabana de Bogotá, una extensa altiplanicie en la Cordillera Oriental de los Andes colombianos. Esta región incluye a Bogotá D.C. y numerosos municipios circundantes.¹

Morfológicamente, representa una altiplanicie situada generalmente entre los 2550 y 2600 metros sobre el nivel del mar (m.s.n.m.), aunque algunos expertos señalan una altura media de 2600 m.s.n.m.⁴, rodeada por sistemas montañosos que alcanzan altitudes superiores.¹

Comprender su extensión geográfica, que varía según las fuentes entre aproximadamente 425,000 hectáreas¹ y 580,000 hectáreas⁷, y sus características topográficas generales es crucial para la selección de fuentes de datos y la escala de visualización. La Sabana de Bogotá es, en esencia, la cuenca alta del río Bogotá.¹

1.3. La Importancia de la Visualización con Estilo de Línea para el Análisis del Terreno

El renderizado basado en líneas ofrece una poderosa abstracción para comprender la estructura del terreno, enfatizando características geomorfológicas, cambios de elevación y particularidades de las pendientes que podrían quedar ocultas por texturas fotorrealistas. Este enfoque puede resaltar elementos topográficos críticos

como crestas, valles, lineamientos tectónicos (identificados en estudios como los de Fierro H y Bernal J A (2008) ⁴) y redes hidrológicas ¹⁰, facilitando diversas tareas analíticas. Por ejemplo, la identificación de lineamientos con dirección NW-SE y SSO a NE-SO, y zonas de cizalla como la Zona de Cizalla del Sur de la Sabana de Bogotá (ZCSB), que influyen en el relieve, pueden ser mejor comunicadas mediante estilos de línea.⁴

1.4. Audiencia Objetivo y Estructura del Informe

Este informe está diseñado para una audiencia técnica, incluyendo gestores de proyectos, especialistas en Sistemas de Información Geográfica (SIG) y desarrolladores web encargados de implementar tal sistema de visualización. Las secciones subsiguientes abordarán sistemáticamente los fundamentos de datos, las elecciones tecnológicas, las técnicas de renderizado, el diseño de la interfaz de usuario, la optimización del rendimiento y concluirán con recomendaciones estratégicas.

La solicitud del usuario de un "estilo de línea" es inherentemente amplia, lo que requiere una exploración exhaustiva de diversas técnicas. Estas pueden variar desde simples mallas de alambre (*wireframes*), que muestran la estructura subyacente de la malla del terreno, hasta complejas líneas generadas proceduralmente y basadas en datos, como curvas de nivel que indican elevaciones constantes, o líneas de pendiente que visualizan la inclinación del terreno. Este informe categorizará estas técnicas, discutiendo sus respectivas complejidades y los resultados visuales que producen, permitiendo una selección informada basada en los objetivos específicos de visualización y los recursos técnicos disponibles.

Además, es fundamental reconocer que la elección de un estilo de línea no es independiente de los datos disponibles ni de la tecnología 3D web seleccionada. Por ejemplo, la generación de curvas de nivel detalladas y precisas depende intrínsecamente de la disponibilidad de Modelos Digitales de Elevación (MDE) exactos y de alta resolución. Similarmente, la facilidad para implementar líneas personalizadas mediante *shaders* (programas que se ejecutan en la GPU) dependerá de las capacidades del motor de renderizado elegido. Algunas bibliotecas pueden ofrecer soporte incorporado para ciertos estilos de línea, mientras que otras podrían requerir un desarrollo más personalizado. Por lo tanto, este informe enfatizará la interconexión entre datos, tecnología y la salida visual deseada, subrayando la necesidad de un enfoque holístico en la planificación del proyecto.

2. Fundamento de Datos: Adquisición y Preparación de un

Modelo 3D de la Sabana de Bogotá

La creación de una visualización 3D precisa y significativa de la Sabana de Bogotá depende fundamentalmente de la calidad y adecuación de los datos geoespaciales subyacentes. Esta sección detalla las fuentes de datos clave, las estrategias para su evaluación y selección, y los procesos necesarios para su preparación y optimización para una visualización web 3D efectiva.

2.1. Identificación de Datos Geoespaciales Clave para la Modelización del Terreno

Para construir un modelo 3D del terreno de la Sabana de Bogotá, se requieren varios tipos de datos geoespaciales:

- **Modelos Digitales de Elevación (MDE):** Este es el conjunto de datos primario.
 - **Instituto Geográfico Agustín Codazzi (IGAC):** Es una fuente principal de datos geoespaciales en Colombia. El IGAC proporciona un MDE derivado de la misión SRTM (Shuttle Radar Topography Mission) con una resolución de 30 metros para todo el país, accesible a través de su plataforma "Colombia en Mapas".¹¹ Además, su portal de "Datos Abiertos"¹³ y su "Biblioteca Virtual"¹³ podrían contener MDEs más específicos, de mayor resolución o datos topográficos detallados para Cundinamarca o la Sabana de Bogotá. Se mencionan ortoimágenes recientes (año 2024) para varios municipios, algunos en Cundinamarca como Zipaquirá, lo que indica una actividad continua en la generación de datos.¹¹
 - **Servicio Geológico Colombiano (SGC):** Ofrece mapas y modelos geológicos, incluyendo el "Mapa Geológico Sabana Bogotá" y un "Modelo Hidrogeológico de la Sabana de Bogotá".¹⁷ Su geoportal es un recurso para acceder a estos productos y potencialmente a MDEs o datos de elevación derivados. El SGC también dispone de mapas geomorfológicos en planchas a escala 1:25.000 que cubren diferentes partes de la Sabana, describiendo unidades geomorfológicas, geoformas, tipos de ladera y características litológicas.¹⁰
 - **Infraestructura de Datos Espaciales para el Distrito Capital (IDECA):** Proporciona un Modelo Digital de Superficie (MDS) para la zona urbana de Bogotá, generado a partir de datos LiDAR.¹⁵ Aunque centrado en lo urbano, representa una fuente de alta resolución para una porción significativa de la Sabana y puede ser útil para análisis detallados en esta área. Estos datos están disponibles para descarga y visualización a través de servicios web geográficos.¹⁵
 - **MDE Globales:** Productos como ASTER GDEM¹⁵ y SRTM¹¹, generalmente con resoluciones de aproximadamente 30 metros, están disponibles globalmente

y pueden servir como línea base o alternativa si no se encuentran datos locales más detallados para toda la extensión.

- **Mapas Topográficos y Curvas de Nivel:**

- La plataforma "Colombia en Mapas" del IGAC lista un "Mapa Topográfico Colombia".²⁶ IDECA ofrece acceso a datos de "Curvas de Nivel" para Bogotá, definidos según el nivel de referencia oficial del IGAC.¹⁵ Estos pueden usarse para validar MDEs o, si se dispone de datos vectoriales, para generar directamente elementos lineales.
- Los mapas geomorfológicos del SGC¹⁰ a menudo incluyen contexto topográfico.

- **Datos Geomorfológicos y Geológicos:**

- El SGC proporciona mapas geomorfológicos detallados (planchas) y estudios⁴ que describen las formas del terreno, pendientes y características estructurales de la Sabana. Esta información es invaluable para interpretar el terreno y, potencialmente, para estilizar las líneas de manera que representen estas características. Por ejemplo, se identifican lineamientos tectónicos y pliegues.⁴
- El IGAC también realiza estudios geomorfológicos y de suelos.¹³ Un informe de gestión de 2019 menciona la interpretación geomorfológica de 223,542.02 ha a escala 1:10.000 en Cundinamarca.¹⁶

- **Datos Hidrológicos:**

- Información sobre el río Bogotá y su llanura de inundación¹⁰ es relevante para visualizar estas características clave. El "Modelo Hidrogeológico de la Sabana de Bogotá" del SGC¹⁷ es un recurso fundamental. La Corporación Autónoma Regional de Cundinamarca (CAR) también publica información sobre la dinámica hidráulica del río Bogotá.¹⁰

- **Delimitación de la Sabana de Bogotá:**

- La Sabana de Bogotá se define generalmente como la cuenca alta del río Bogotá.¹ Su extensión varía entre 425,000 hectáreas¹ y 580,000 hectáreas.⁷ Comprende Bogotá D.C. y municipios circundantes como Chía, Cota, Funza, Mosquera, Soacha, Facatativá, Zipaquirá, Tabio, Tenjo, entre otros.² Esta definición es crítica para la extracción y el alcance de los datos.

2.2. Estrategia de Evaluación y Selección de Datos

La selección de los datos adecuados es un paso crítico que impactará directamente la calidad y el rendimiento de la visualización.

- **Resolución vs. Rendimiento:** Los MDE de mayor resolución (por ejemplo, los derivados de LiDAR para Bogotá¹⁵) proporcionan mayor detalle, pero resultan en

conjuntos de datos más grandes, lo que puede afectar el rendimiento en la web. Es necesario encontrar un equilibrio. Para un área extensa como la Sabana, un MDE de 30 metros (como el SRTM del IGAC ¹¹) podría ser un punto de partida práctico, con la opción de incorporar inserciones de mayor resolución en áreas localizadas si están disponibles y son necesarias.

- **Cobertura y Consistencia:** Se debe asegurar que el MDE elegido ofrezca una cobertura consistente para toda la región definida de la Sabana de Bogotá.
- **Precisión Vertical y Unidades:** Es fundamental verificar la precisión vertical y las unidades (generalmente metros) del MDE.
- **Formato de Datos:** GeoTIFF es un formato común para MDEs.³⁰ Se debe asegurar la compatibilidad con las herramientas de procesamiento y las bibliotecas 3D elegidas.
- **Revisión de Metadatos:** Se debe realizar una revisión exhaustiva de los metadatos de los conjuntos de datos seleccionados ¹⁰ para comprender su origen, procesamiento y limitaciones. Por ejemplo, los metadatos de las planchas geomorfológicas del SGC pueden indicar la escala y el tipo de información contenida.¹⁰

2.3. Procesamiento y Optimización de Datos para Visualización Web 3D

Una vez seleccionados, los datos crudos deben ser procesados y optimizados para su uso en un entorno web 3D.

- **Preparación del MDE:**
 - **Reproyección:** Si es necesario, los MDE deben ser reproyectados a un sistema de coordenadas geográficas consistente (por ejemplo, WGS84, EPSG:4326, como lo hace automáticamente Cesium ion ³⁰) o a un sistema proyectado adecuado para la región si las mediciones locales son críticas.
 - **Mosaico:** Si múltiples teselas de MDE cubren la Sabana, necesitarán ser unidas (mosaico) en un único ráster.
 - **Recorte:** El MDE debe ser recortado a los límites precisos de la Sabana de Bogotá.
 - **Manejo de Valores Sin Datos (No-Data):** Asegurar que los valores sin datos se manejen correctamente para evitar artefactos en el modelo 3D.
- **Teselado del Terreno para Streaming Web:**
 - **Quantized Mesh:** Para bibliotecas como CesiumJS, los MDEs se convierten típicamente en teselas de Quantized Mesh.³⁰ Herramientas como tin-terrain ³⁶ o servicios como Cesium ion ³⁰ pueden realizar esta conversión desde GeoTIFF. GISBox es otra herramienta mencionada para este propósito.⁴³ Este formato está optimizado para la transmisión eficiente de mallas de terreno.

- **Mapas de Altura (*Heightmaps*) para Otros Motores:** Para bibliotecas como Three.js o Babylon.js, el MDE podría procesarse en un formato de imagen de mapa de alturas (por ejemplo, PNG en escala de grises) que el motor puede usar para desplazar una malla plana y así generar el terreno.³¹
- **Optimización de GeoTIFFs (si se usan directamente o como intermedios):**
 - Técnicas como el uso de compresión adecuada (por ejemplo, LZW), el teselado dentro del GeoTIFF y la creación de vistas generales (*overviews*) pueden optimizar el tamaño del archivo y la velocidad de acceso.³⁴ Sin embargo, para la transmisión web, generalmente se prefiere la conversión a formatos teselados especializados.
- **Preparación de Datos Vectoriales (por ejemplo, para curvas de nivel predefinidas):**
 - Si se utilizan curvas de nivel vectoriales existentes (por ejemplo, de archivos shapefile o GeoJSON), se debe asegurar que estén en un formato amigable para la web (GeoJSON es común³⁵). Es importante simplificar la geometría si es necesario para reducir el tamaño del archivo y mejorar el rendimiento del renderizado, especialmente para áreas grandes.

La calidad y naturaleza de la visualización de líneas están fundamentalmente limitadas por el MDE de entrada. Los estilos de línea abstractos, como las líneas de pendiente o las líneas de cresta, requieren cálculos robustos de derivadas (pendiente, aspecto, curvatura) a partir del MDE. Si el MDE es ruidoso o de baja resolución, estas líneas derivadas serán inexactas o carecerán de significado. Por lo tanto, se debe invertir un esfuerzo significativo en la obtención, evaluación y preparación del mejor MDE posible antes de intentar la generación de estilos de línea complejos. Esto podría implicar la comparación de diferentes MDE (por ejemplo, SRTM del IGAC frente a posibles datos del SGC o ASTER GDEM) para la Sabana de Bogotá, considerando sus respectivas precisiones y resoluciones.

Dadas las diferentes resoluciones de los MDE disponibles (por ejemplo, LiDAR de alta resolución para la ciudad de Bogotá¹⁵ frente a MDE regionales más gruesos¹¹), se podría considerar un modelo de terreno multirresolución. Esto implicaría utilizar datos de mayor resolución donde estén disponibles y fusionarlos con datos de menor resolución para el resto de la Sabana. Este enfoque optimiza tanto la calidad visual en áreas clave como el rendimiento general, y las bibliotecas 3D web a menudo admiten mecanismos de Nivel de Detalle (LOD) esenciales para esta tarea.

Finalmente, la rica información geomorfológica disponible del SGC⁴ e IGAC¹³ para la Sabana, incluyendo detalles sobre geoformas, pendientes y lineamientos tectónicos⁴, no debe verse solo como complementaria. Esta información puede informar

directamente los *tipos* de estilos de línea a implementar y *dónde* aplicarlos para obtener el máximo valor analítico, por ejemplo, resaltando fallas geológicas o límites específicos de geoformas. Esto elevaría la visualización de un simple modelo 3D a una herramienta más analítica.

3. Selección de la Pila Tecnológica Web 3D Óptima

La elección de la biblioteca o *framework* JavaScript 3D adecuado es una decisión fundamental que influirá en la facilidad de desarrollo, las capacidades de visualización, el rendimiento y la mantenibilidad del proyecto. A continuación, se presenta una visión general de las principales bibliotecas, los criterios de evaluación y una comparación para guiar esta selección.

3.1. Panorama de las Principales Bibliotecas JavaScript 3D/Geoespaciales

Existen varias bibliotecas JavaScript maduras y potentes para la creación de experiencias 3D en la web, cada una con sus fortalezas y enfoques:

- **CesiumJS:** Es una biblioteca JavaScript de código abierto diseñada específicamente para la visualización geoespacial 3D en un globo terráqueo WGS84. Destaca por su capacidad para transmitir conjuntos de datos masivos, incluyendo terreno y 3D Tiles.⁴⁷ Admite diversos formatos de datos geoespaciales y posee sólidas capacidades para la visualización dinámica en el tiempo. Su enfoque geoespacial nativo y el robusto renderizado de terreno²⁴, junto con el soporte para Quantized Mesh³⁰, la convierten en una candidata principal.
- **Three.js:** Es una biblioteca de gráficos 3D muy popular y versátil para WebGL. Aunque no es inherentemente geoespacial, es altamente extensible y puede adaptarse para el renderizado de terreno y datos geoespaciales con esfuerzo adicional o mediante bibliotecas complementarias (por ejemplo, three-geospatial⁵²). Es conocida por su flexibilidad y su amplia comunidad.⁵³ Su flexibilidad permite efectos de renderizado y *shaders* altamente personalizados.³¹ Los archivos GeoTIFF pueden procesarse en mapas de altura para la generación de terreno.³¹
- **Babylon.js:** Es otro potente *framework* WebGL de código abierto para construir juegos y aplicaciones 3D. Cuenta con un rico conjunto de herramientas, incluyendo un Editor de Materiales Nodales, y es conocido por su facilidad de uso y rendimiento.⁵⁴ Puede utilizarse para aplicaciones de terreno y geoespaciales.⁶⁵ Ofrece sólidas capacidades de renderizado y herramientas para efectos personalizados; sus características de terreno dinámico⁷⁰ podrían ser relevantes.
- **Deck.gl:** Es un *framework* impulsado por WebGL para el análisis exploratorio visual de grandes conjuntos de datos. Sobresale en visualizaciones geoespaciales

2D y 3D, a menudo utilizado con mapas base como Mapbox GL JS o Google Maps. Proporciona una capa TerrainLayer⁷⁵ y una capa ContourLayer.⁷⁷ La TerrainLayer maneja directamente los datos de elevación para la reconstrucción de mallas, y la ContourLayer puede generar isolíneas. Su enfoque en la visualización de datos es pertinente.

3.2. Criterios de Evaluación para la Visualización de la Sabana de Bogotá

La selección de la biblioteca más adecuada para este proyecto debe basarse en los siguientes criterios:

- **Manejo Nativo de MDE/Terreno:** ¿Con qué facilidad la biblioteca ingiere, procesa y renderiza datos MDE (por ejemplo, de GeoTIFFs o formatos teselados como Quantized Mesh)? CesiumJS y la TerrainLayer de Deck.gl tienen un fuerte soporte nativo. Three.js y Babylon.js requieren una configuración más manual o extensiones.
- **Precisión Geoespacial y Proyecciones:** ¿Maneja la biblioteca con precisión los sistemas de coordenadas geoespaciales y las proyecciones? CesiumJS está construido sobre WGS84. Otras podrían requerir bibliotecas externas o transformaciones personalizadas.
- **Capacidades de Renderizado de Líneas:**
 - **Malla de Alambre (Wireframe):** Facilidad para renderizar mallas de alambre básicas del terreno. La mayoría de las bibliotecas ofrecen esto.⁴⁶
 - **Líneas Vectoriales:** Capacidad para renderizar y proyectar (*drape*) líneas vectoriales (por ejemplo, curvas de nivel GeoJSON) sobre el terreno. CesiumJS³⁰, Deck.gl⁴⁸, y Three.js (con esfuerzo³¹) pueden hacerlo.
 - **Líneas Procedurales (Shaders):** Soporte para *shaders* GLSL personalizados para generar líneas basadas en propiedades del terreno (altura, pendiente, curvatura). Todas las principales bibliotecas WebGL (CesiumJS⁸²; Three.js⁴⁵; Babylon.js⁶⁷) lo permiten.
- **Rendimiento con Grandes Conjuntos de Datos:** La Sabana es un área extensa. La biblioteca debe manejar eficientemente mallas de terreno grandes y potencialmente muchas geometrías de línea, utilizando técnicas como el Nivel de Detalle (LOD).²⁴ CesiumJS es particularmente fuerte en este aspecto debido a su arquitectura de teselado y transmisión (*streaming*).
- **Facilidad de Integración y Extensibilidad:** ¿Qué tan bien se puede integrar la biblioteca en una aplicación web personalizada? ¿Qué tan fácil es extenderla con funcionalidades personalizadas?
- **Soporte Comunitario y Documentación:** Una comunidad fuerte y una documentación exhaustiva son vitales para el desarrollo. Three.js y CesiumJS

tienen comunidades grandes y activas.

- **Adecuación para la Visualización de la "Sabana":** Ajuste general a los requisitos específicos de visualización de la Sabana de Bogotá con estilos de línea.

3.3. Tabla: Comparación de Bibliotecas Web 3D para Visualización Geoespacial

Característica	CesiumJS	Three.js	Babylon.js	Deck.gl
Manejo Nativo MDE/Terreno	Excelente (Quantized Mesh, proveedores de terreno)	Manual (Mapas de altura, geometría personalizada) / Extensiones	Manual (Mapas de altura, Terreno Dinámico) / Extensiones	Bueno (TerrainLayer a partir de imágenes de mapa de alturas)
Proyecciones Geoespaciales	Globo WGS84 nativo, buen soporte de proyecciones	Requiere bibliotecas externas (ej. Proj4js) o matemática personalizada	Requiere bibliotecas externas o matemática personalizada	Se integra con proveedores de mapas, maneja proyecciones
Integración GeoTIFF/MDE	Excelente vía Cesium ion o herramientas como tin-terrain ³⁰	Procesamiento manual a mapa de altura/geometría ³¹	Procesamiento manual a mapa de altura/geometría ⁷⁰	TerrainLayer ingiere imágenes de mapa de alturas ⁷⁵
Renderizado de Wireframe	Flags de depuración para 3DTilesets ⁷⁸ , Material para Globo ⁷⁸	Propiedad de material ⁵⁷	Propiedad de material ⁸⁰	Prop wireframe en SimpleMeshLayer (vía TerrainLayer ⁷⁵)
Proyección Líneas Vectoriales	Excelente (<i>clamping</i> GeoJSON, KML ³⁰)	Posible con <i>raycasting</i> /lógica personalizada ³¹	Posible con <i>raycasting</i> /lógica personalizada	Bueno (GeoJsonLayer, PathLayer)
Líneas Procedurales	Excelente (Fabric, Custom)	Excelente (ShaderMaterial)	Excelente (Editor)	Limitado a parámetros de

(Shaders)	<i>Shaders</i> ⁸²⁾	⁴⁵⁾	Materiales Nodales, <i>Shaders</i> ⁶⁷⁾	capa, menos control directo de <i>shader</i>
Rendimiento (Grandes Datos)	Excelente (LOD, <i>streaming</i>) ²⁴	Depende de implementación, requiere optimización ⁵⁹	Depende de implementación, requiere optimización ⁷²	Bueno para datos agregados, TerrainLayer usa teselas
Comunidad/Ecosistema	Fuerte, enfoque geoespacial	Muy grande, 3D general	Creciente, enfoque juegos/aplicaciones	Fuerte, enfoque visualización de datos
Adecuación Viz. Sabana	Alta (Nativo geoespacial)	Media-Alta (Flexible, más trabajo personalizado)	Media (Buen renderizado, más trabajo geo personalizado)	Media-Alta (Bueno para capas de datos, terreno basado en imágenes)

Esta tabla proporciona una comparación estructurada y directa de las características más relevantes de cada biblioteca, facilitando una selección tecnológica más informada y eficiente.

Es importante considerar el balance entre la especialización geoespacial y la flexibilidad de propósito general. Bibliotecas como CesiumJS son inherentemente geoespaciales, ofreciendo soporte inmediato para terreno global, teselado y coordenadas geoespaciales.³⁰ Esto puede acelerar el desarrollo para aplicaciones puramente geoespaciales. Por otro lado, bibliotecas 3D de propósito general como Three.js o Babylon.js ofrecen una inmensa flexibilidad para renderizado personalizado pero requieren más esfuerzo para integrar y gestionar datos geoespaciales con precisión.⁵³ Implementar funcionalidad geoespacial en estas últimas exige código personalizado o extensiones.⁵² Deck.gl se sitúa en un punto intermedio, con fortalezas en capas de datos geoespaciales, pero su TerrainLayer depende de imágenes de mapa de alturas y podría no ser tan flexible para fuentes MDE arbitrarias o efectos de terreno procedurales avanzados como un motor 3D completo.⁷⁵ La elección óptima dependerá del énfasis principal del proyecto: un contexto global preciso versus una visualización local altamente personalizada.

Más allá de las características técnicas, la "experiencia del desarrollador" es un factor crucial. La facilidad de uso, la calidad de la documentación (por ejemplo, controles de

cámara en Three.js ⁹⁰; cámara en CesiumJS ⁹²; cámara en Babylon.js ⁶⁸), el soporte comunitario (foros como ⁹⁴) y los recursos de aprendizaje disponibles (tutoriales como ³¹ para Three.js³⁰ para CesiumJS) impactan significativamente la velocidad de desarrollo y la capacidad para superar desafíos. Una biblioteca técnicamente superior con documentación deficiente o una comunidad pequeña puede ralentizar el progreso. Por lo tanto, la evaluación debe considerar la experiencia general que cada biblioteca ofrece al desarrollador.

4. Dominando las Técnicas de Visualización con Estilo de Línea para Terreno

Una vez establecida la base de datos y seleccionada la pila tecnológica, el siguiente paso es implementar las técnicas de renderizado de líneas. Esta sección explora varias aproximaciones, desde mallas de alambre básicas hasta efectos de líneas estilizadas mediante *shaders* GLSL personalizados.

4.1. Implementación del Renderizado de Malla de Alambre (*Wireframe*)

- **Concepto:** Visualiza la geometría subyacente de la malla del terreno dibujando sus aristas. Esto revela la estructura de triángulos que compone la superficie 3D.
- **CesiumJS:**
 - Para Cesium3DTileset, la propiedad `debugWireframe` puede establecerse en `true`.⁷⁸ Esto está destinado principalmente a la depuración de 3D Tiles.
 - Para el terreno principal del globo, no existe una propiedad directa de *wireframe*. Se puede lograr un efecto similar aplicando un material `Polyline` personalizado a `viewer.scene.globe.material` y estableciendo `globe.fill = false`;⁷⁸ Este enfoque simula un *wireframe* dibujando líneas basadas en la interpretación de la geometría por parte del material. La comunidad ha discutido opciones para superponer *wireframes* sobre texturas.⁸⁹
- **Three.js:**
 - Materiales como `MeshBasicMaterial` o `MeshStandardMaterial` tienen una propiedad `wireframe: true`.³¹ Esto es sencillo de aplicar a una malla de terreno.
 - También se puede usar `THREE.WireframeGeometry` o `THREE.EdgesGeometry` con `THREE.LineSegments` para un mayor control.³¹
- **Babylon.js:**
 - Los materiales (por ejemplo, `StandardMaterial`) tienen una propiedad `wireframe: true`.⁸⁰
 - También se puede usar `mesh.enableEdgesRendering()` para mostrar aristas con color y grosor controlables.⁷³
- **Deck.gl:**

- La TerrainLayer reenvía su propiedad wireframe a una SimpleMeshLayer interna.⁷⁵ Establecer wireframe: true en la TerrainLayer debería renderizar la malla del terreno en modo *wireframe*.
- La GeoJsonLayer también tiene una opción wireframe para polígonos extruidos.⁴⁸
- **Consideraciones:** Los *wireframes* básicos pueden volverse muy densos y visualmente abrumadores en mallas de alta resolución. Para terrenos extensos, podría ser necesario un renderizado de *wireframe* consciente del Nivel de Detalle (LOD) o un ajuste dinámico de la densidad.

4.2. Generación y Visualización de Curvas de Nivel

- **Concepto:** Líneas que conectan puntos de igual elevación, proporcionando una representación clara de la forma y la pendiente del terreno.
- **Método 1: Generación Procedural mediante Shaders (Tiempo Real):**
 - **CesiumJS:**
 - El tipo de material ElevationContour en Fabric puede renderizar curvas de nivel proceduralmente sobre el globo.⁸² Los parámetros incluyen spacing (espaciado), width (ancho) y color.
 - Se pueden combinar múltiples materiales ElevationContour usando *shaders* GLSL personalizados dentro de Fabric para crear curvas de nivel mayores y menores con diferentes estilos.⁸⁷
 - **Three.js/Babylon.js:** Se pueden escribir *shaders* GLSL personalizados para calcular y renderizar curvas de nivel directamente en la GPU. Esto implica determinar la elevación en el *fragment shader* y dibujar una línea si cae en un intervalo de curva de nivel. Técnicas como el uso de fwidth() (requiere la extensión OES_standard_derivatives⁹⁸) pueden ayudar a crear líneas nítidas y con anti-aliasing.²⁵
 - **Rendimiento:** Las curvas de nivel procedurales pueden ser muy eficientes ya que se generan en la GPU y se adaptan a la vista actual y al LOD.
- **Método 2: Pre-cálculo y Renderizado Vectorial (Marching Squares):**
 - **Algoritmo:** El algoritmo Marching Squares se puede aplicar a la cuadrícula 2D del MDE para extraer curvas de nivel como datos vectoriales (polilíneas).⁷⁴
 - **Bibliotecas JavaScript:** Bibliotecas como MarchingSquares.js¹⁰⁰ o d3-contour¹⁰¹ pueden realizar este cálculo en el lado del cliente o del servidor.
 - **Renderizado:** Las curvas de nivel vectoriales generadas (por ejemplo, en formato GeoJSON) pueden luego renderizarse y proyectarse sobre el terreno 3D.
 - **CesiumJS:** Usar GeoJsonDataSource o KmlDataSource y fijar las entidades al terreno (*clamp to ground*).³⁰

- **Three.js/Babylon.js:** Requiere lógica personalizada para analizar GeoJSON, crear geometrías de línea y ajustar las alturas de los vértices para que se ajusten a la malla del terreno.³¹
- **Deck.gl:** La *ContourLayer* puede procesar directamente datos de puntos (con la elevación como peso) para generar y renderizar isolíneas usando *Marching Squares* internamente.⁷⁷ También puede tomar datos de líneas GeoJSON mediante *GeoJsonLayer*.⁴⁸
- **Rendimiento:** La generación de curvas de nivel a partir de un MDE grande en el lado del cliente puede ser computacionalmente intensiva. Se recomienda el pre-cálculo en un servidor o el uso de una biblioteca del lado del cliente eficiente con optimizaciones (como *QuadTree* en *MarchingSquares.js* ¹⁰⁰). Renderizar un gran número de líneas vectoriales también puede afectar el rendimiento; pueden ser necesarias estrategias de simplificación y LOD para las líneas vectoriales.¹⁰⁵
- **Estilización:** Control del color de línea, ancho, discontinuidad (*dashing*) y potencialmente etiquetas (aunque el etiquetado en WebGL es complejo ¹⁰⁷).

4.3. Efectos de Línea Avanzados y Estilizados usando *Shaders GLSL* Personalizados

- **Concepto:** Utilizar *shaders GLSL* para crear representaciones de líneas más abstractas o artísticas basadas en características del terreno que van más allá de la simple elevación.
- **Líneas de Pendiente / Jaspeado (*Hatching*):**
 - Líneas dibujadas en la dirección de la pendiente más pronunciada. Su densidad o grosor puede representar la magnitud de la pendiente.¹⁰⁸
 - Requiere calcular las normales y gradientes del terreno en el *shader*.
 - **Implementación:** Posible en CesiumJS (*Fabric/CustomShader* ⁸⁴), Three.js (*ShaderMaterial* ⁴⁵), Babylon.js (*Custom Shaders/NME* ⁶⁷).
- **Líneas de Cresta y Valle:**
 - Líneas que resaltan las cimas de las crestas y los fondos de los valles.
 - Requiere calcular la curvatura del terreno (por ejemplo, curvaturas principales ⁴⁵) en el *shader* o pre-procesar esta información en texturas.
- **Contornos Sugestivos / Crestas Aparentes:**
 - Líneas que capturan características de la forma basadas en la dirección de la vista y la curvatura de la superficie, a menudo extendiendo las siluetas.²⁵
 - Pueden implementarse en el espacio de la imagen usando *shaders* mediante el análisis de las derivadas del sombreado dependiente de la vista.
- **Líneas de Flujo:**
 - Líneas que simulan el flujo de agua u otros fenómenos sobre el terreno.⁸⁵

- A menudo implican sistemas de partículas o la animación de texturas a lo largo de trayectorias derivadas de la pendiente.
- **Enfoque General:**
 - **Vertex Shader:** Acceder a la altura del terreno, calcular normales, pasar datos relevantes (por ejemplo, altura, normal, coordenadas UV) al *fragment shader*.
 - **Fragment Shader:**
 - Calcular derivadas (por ejemplo, dFdx, dFdy con OES_standard_derivatives⁹⁸) para determinar la pendiente, el aspecto o la curvatura.
 - Usar lógica condicional (if, step, smoothstep) para determinar dónde deben dibujarse las líneas en función de estos cálculos.
 - Aplicar color y estilo a las líneas.
 - Se pueden encontrar ejemplos de GLSL para coloreado de terreno y efectos en tutoriales como.⁴⁵
- **Materiales Personalizados de CesiumJS:** El sistema Fabric⁸³ y la API CustomShader⁸⁴ permiten inyectar código GLSL para modificar materiales existentes o crear nuevos. Esta es la forma principal de lograr tales efectos en el globo de Cesium.

4.4. Proyección de Datos de Línea 2D (por ejemplo, GeoJSON) sobre Mallas de Terreno 3D

- **Concepto:** Tomar geometrías de línea 2D definidas externamente (por ejemplo, curvas de nivel precalculadas, carreteras, ríos de un SIG) y renderizarlas con precisión sobre la superficie del terreno 3D.
- **CesiumJS:** Admite directamente la carga de GeoJSON, KML y CZML. Las entidades pueden fijarse al suelo, proyectándolas efectivamente.³⁰ Para polilíneas, esto implica muestrear las alturas del terreno a lo largo de los segmentos de la línea.
- **Three.js/Babylon.js:**
 - Requiere analizar el GeoJSON (u otro formato vectorial).
 - Para cada vértice en la línea 2D, es necesario determinar su posición 3D en la malla del terreno. Esto típicamente implica:
 1. Convertir coordenadas geográficas (lat, lon) a las coordenadas del mundo 3D del modelo de terreno.
 2. Realizar un lanzamiento de rayos (*raycasting*) desde arriba del terreno hacia abajo en las coordenadas del mundo (x, y) para encontrar el punto de intersección con la malla del terreno, obteniendo así el valor z (altura) correcto.

3. Crear una nueva geometría de línea 3D con estos vértices ajustados.
 - El tutorial ³¹ (Three.js con GeoTIFF) crea la malla del terreno; la proyección de líneas sería un paso adicional. Se necesitarían bibliotecas o código personalizado para el *raycasting* contra la malla. ⁴⁹ menciona métodos para georreferenciar y proyectar vértices de modelos. ³¹ (terreno Three.js desde GeoTIFF) se centra en la creación del terreno; la proyección de líneas es un problema aparte.
- **Deck.gl:** La GeoJsonLayer puede renderizar geometrías LineString. Cuando se usa junto con la TerrainLayer, Deck.gl maneja el muestreo de elevación para proyectar las líneas, aunque los detalles específicos de esta interacción necesitarían ser verificados en su documentación.

4.5. Tabla: Resumen de Técnicas de Renderizado con Estilo de Línea

Técnica	Descripción	Pros	Contras	Complejidad Implementación (Típica)	Resultado Visual	Adecuación Modelo Sabana
Wireframe Básico	Renderiza todas las aristas de la malla del terreno.	Simple de implementar; muestra geometría subyacente.	Puede ser muy denso/ruidoso en mallas detalladas; puede ocultar características superficiales.	Baja (a menudo un flag de material ⁵⁷)	Estructural, similar a una cuadrícula.	Bueno para depuración o vista técnica, menos para estética.
Curvas de Nivel (Procedural/Shader)	Líneas de igual elevación generadas en la GPU.	Eficiente; dinámico con LOD; líneas nítidas posibles. ⁸²	Complejidad del <i>shader</i> ; calidad depende del MDE y lógica del <i>shader</i> .	Media a Alta (GLSL ⁸³)	Líneas de elevación suaves y adaptativas.	Excelente para representación topográfica.

Curvas de Nivel (Marching Squares + Vector)	MDE procesado para extraer curvas de nivel vectoriales, luego renderizadas.	Técnica SIG estándar; puede usar datos vectoriales existentes. ⁷⁷	Generación en cliente puede ser lenta; muchos vectores pueden afectar rendimiento. ¹⁰²	Media (Algoritmo + Renderizado Vectorial ⁴⁹⁾)	Líneas de elevación discretas.	Bueno si está pre-procesado o para curvas estáticas.
Líneas Pendiente/Aspecto (Shader)	Líneas que indican dirección/magnitud de pendiente o aspecto.	Resalta características geomorfológicas; basado en datos. ¹⁰⁸	Complejidad del <i>shader</i> ; requiere normales/derivadas de buena calidad.	Alta (GLSL)	Líneas estilizadas y analíticas.	Muy bueno para análisis geomorfológico detallado.
Contornos Sugestivos (Shader)	Líneas dependientes de la vista que resaltan la forma más allá de las siluetas.	Mejora la percepción de la forma; artístico. ²⁵	Lógica de <i>shader</i> compleja; computacionalmente intensivo.	Muy Alta (GLSL)	Líneas expresivas que definen la forma.	Potencialmente muy efectivo para descripción matizada.
Líneas GeoJSON Proyectadas	Renderiza líneas vectoriales 2D existentes (ej. ríos, carreteras) sobre terreno 3D.	Usa datos externos autorizados; clara representación de características. ³⁰	Rendimiento depende de complejidad línea; requiere proyección precisa.	Media (Carga datos + Proyección ⁴⁹⁾)	Características geográficas contextuales.	Esencial para añadir contexto como ríos o infraestructura.

Esta tabla compara sistemáticamente diferentes técnicas de renderizado de líneas, destacando sus características, ventajas, desventajas, complejidad y adecuación para la visualización de la Sabana de Bogotá.

Existe una dicotomía fundamental para características como las curvas de nivel: generarlas en tiempo real usando *shaders* (se adapta al LOD, intensivo en GPU) o pre-procesarlas (por ejemplo, usando Marching Squares) en datos vectoriales para luego proyectarlas (intensivo en CPU/memoria para grandes conjuntos de datos, líneas estáticas). La elección impacta el rendimiento, el dinamismo y la complejidad de la implementación. La generación mediante *shaders* en tiempo real ⁵⁵ descarga el trabajo a la GPU y puede ser muy eficiente para la visualización, pero la lógica del *shader* puede ser compleja. El pre-procesamiento crea datos vectoriales estáticos; renderizar muchas líneas vectoriales puede ser un cuello de botella si no se optimiza.⁷⁷ La decisión depende de si el dinamismo (por ejemplo, cambiar el intervalo de las curvas sobre la marcha) es crítico y de las características de rendimiento de los dispositivos cliente.

La visualización más efectiva podría no depender de un único estilo de línea, sino de una combinación. Por ejemplo, un *wireframe* base + curvas de nivel mayores/menores + un jaspeado de pendiente en áreas específicas. Esto requiere un motor de renderizado y un enfoque que admita la superposición o composición de estos efectos.⁸⁹ La implementación de efectos combinados requiere una cuidadosa consideración del pipeline de renderizado, lo que podría implicar renderizado multipaso, *shaders* complejos que calculan y mezclan múltiples efectos de línea, o la superposición de diferentes tipos de geometría. El soporte de la biblioteca 3D elegida para tales sistemas de composición o materiales avanzados (como Fabric de Cesium ⁸⁷) se vuelve crítico.

Finalmente, algunos estilos de línea son principalmente analíticos (por ejemplo, curvas de nivel precisas ⁸²), mientras que otros pueden ser más artísticos o sugestivos (por ejemplo, contornos sugestivos ¹¹¹, jaspeado estilizado ¹⁰⁸). La elección depende del objetivo principal de la visualización: medición/análisis preciso o comunicación intuitiva de la forma. Los *wireframes* ⁷⁸ son estructurales y analíticos. El informe debe guiar al usuario a considerar si su objetivo principal es el análisis cuantitativo o la comprensión cualitativa de la forma, lo que influye en la elección de algoritmos y técnicas de renderizado.

5. Diseño de una Interfaz Gráfica Web (GUI) Efectiva

Una GUI bien diseñada es crucial para permitir a los usuarios interactuar de manera intuitiva con el modelo 3D de la Sabana de Bogotá y sus diversas representaciones de líneas. La interfaz debe ser potente pero fácil de usar, equilibrando la riqueza de funciones con la simplicidad.

5.1. Componentes Centrales de la Interfaz de Usuario

La GUI debe incluir los siguientes componentes esenciales:

- **Visor 3D Principal:** El área central donde se renderiza el modelo de la Sabana. Ocupará la mayor parte de la pantalla.
- **Barra de Herramientas/Panel de Control:** Contendrá botones, selectores y otros controles para acceder a las opciones de visualización, selección de estilos de línea, gestión de capas y otras herramientas. Podría ubicarse en la parte superior, lateral o ser un panel flotante.
- **Visualización de Información:** Un área discreta para mostrar información contextual, como las coordenadas geográficas y la elevación del punto bajo el cursor del ratón, o información sobre elementos seleccionados.
- **Indicadores de Carga:** Elementos visuales (por ejemplo, barras de progreso, *spinners*) para proporcionar retroalimentación al usuario durante la carga y el procesamiento de datos, especialmente para un modelo extenso como el de la Sabana.

5.2. Controles de Cámara y Navegación Esenciales

Una navegación fluida e intuitiva es fundamental para la exploración 3D:

- **Órbita:** Permitir al usuario rotar la cámara alrededor de un punto de destino o del centro de la vista actual. Bibliotecas como Three.js (OrbitControls ⁹⁰), CesiumJS (controles predeterminados ⁹²) y Babylon.js (ArcRotateCamera ⁶⁸) proporcionan implementaciones robustas de este control.
- **Desplazamiento Lateral (*Pan*):** Mover la cámara horizontal y verticalmente a través de la escena sin cambiar la orientación o el zoom.
- **Zoom (*Acercar/Alejar*):** Mover la cámara más cerca o más lejos de la escena (efecto *dolly*). Generalmente se controla con la rueda del ratón o gestos táctiles.
- **Inclinación (*Tilt/Pitch*):** Ajustar el ángulo de visión de la cámara con respecto al horizonte, permitiendo vistas desde arriba hacia abajo o perspectivas más oblicuas.
- **Restablecer Vista:** Un botón o función para devolver la cámara a una vista predeterminada o inicial de la Sabana.
- **Volar a Ubicación (*Fly-to*):** Funcionalidad para animar suavemente la cámara hacia puntos de interés específicos o resultados de búsqueda dentro de la Sabana. CesiumJS, por ejemplo, tiene capacidades *flyTo* bien desarrolladas.⁹³

5.3. Controles para Estilos de Línea y Capas de Datos

La interfaz debe permitir al usuario controlar la apariencia de las líneas y gestionar las

capas de datos:

- **Conmutadores de Estilo de Línea:** Casillas de verificación o botones de radio para habilitar o deshabilitar diferentes estilos de línea (por ejemplo, *Wireframe*, Curvas de Nivel, Líneas de Pendiente).
- **Personalización de Curvas de Nivel (si aplica):**
 - Entrada para el intervalo o espaciado de las curvas de nivel (por ejemplo, el uniforme spacing del material *ElevationContour* de CesiumJS ⁸³).
 - Controles para el color y el grosor de las líneas.
- **Gestión de Capas:** Si se utilizan múltiples capas de datos (por ejemplo, terreno base, datos vectoriales proyectados, diferentes resoluciones de MDE), se necesita un control de capas para gestionar la visibilidad y la opacidad. CesiumJS ofrece una gestión robusta de capas de imágenes y datos.⁵¹

5.4. Elementos Interactivos para la Exploración de Datos

Para facilitar un análisis más profundo, se pueden incluir elementos interactivos:

- **Consulta de Elevación Basada en el Ratón:** Mostrar las coordenadas geográficas (latitud, longitud) y la elevación del punto del terreno bajo el cursor del ratón. Esto se puede lograr con `sampleTerrainMostDetailed` en CesiumJS ³⁰ o mediante *raycasting* en *Three.js/Babylon.js*.
- **Funcionalidad de Búsqueda:** Permitir a los usuarios buscar municipios específicos, puntos de referencia o coordenadas dentro de la Sabana y hacer que la cámara navegue hasta allí.
- **Herramientas de Medición (Opcional):** Herramientas para medir distancias o áreas sobre el terreno 3D.

5.5. Consideraciones de Experiencia de Usuario (UX)

Una buena UX es clave para la adopción y utilidad de la herramienta:

- **Navegación Intuitiva:** Los controles deben ser estándar y fáciles de aprender. La respuesta a las acciones del ratón y teclado debe ser predecible.
- **Capacidad de Respuesta:** La interfaz debe responder rápidamente a las interacciones del usuario, sin retardos notables.
- **Claridad:** Los elementos visuales y los controles deben estar claramente etiquetados y ser comprensibles. El uso de iconos consistentes y tooltips puede ayudar.
- **Accesibilidad:** Considerar los principios básicos de accesibilidad en el diseño de la GUI, como el contraste de color adecuado y la navegabilidad por teclado para los controles.

Los controles específicos de la interfaz de usuario necesarios dependerán en gran medida de los estilos de línea implementados. Por ejemplo, los *shaders* de curvas de nivel procedurales podrían permitir el ajuste dinámico de los intervalos de las curvas mediante un deslizador, actualizando los uniformes del *shader* en tiempo real.⁸² Esto permite una exploración dinámica de los patrones de las curvas. En cambio, las curvas de nivel vectoriales pre-procesadas podrían simplemente tener un conmutador de visibilidad y quizás algunas opciones básicas de estilo. Si se utilizan efectos de *shader* avanzados, como líneas de pendiente, los controles de la interfaz de usuario podrían permitir a los usuarios establecer umbrales para la visualización de pendientes o cambiar las rampas de color. Por lo tanto, el diseño de la interfaz de usuario no puede finalizarse hasta que se elijan las técnicas específicas de renderizado de líneas.

Si bien son posibles muchas características interactivas, una interfaz de usuario sobrecargada puede resultar abrumadora. El diseño debe priorizar los controles esenciales para el objetivo principal (visualización del terreno con estilo de línea) y considerar la revelación progresiva para opciones más avanzadas. Una interfaz inicial limpia y centrada en las tareas principales, con opciones avanzadas en submenús o paneles de "configuración avanzada", asegura que la herramienta sea accesible para usuarios ocasionales y al mismo tiempo proporcione profundidad para usuarios expertos.

6. Asegurando un Rendimiento Óptimo para la Visualización de Terrenos a Gran Escala

La visualización de un área extensa y topográficamente compleja como la Sabana de Bogotá en un entorno web 3D exige una cuidadosa atención al rendimiento. Sin optimización, la aplicación podría volverse lenta e inutilizable, especialmente en dispositivos con recursos limitados.

6.1. Estrategias de Gestión del Nivel de Detalle (LOD)

- **LOD del Terreno:**
 - **Concepto:** Renderizar el terreno con diferentes niveles de detalle según la distancia a la cámara. El terreno más cercano se renderiza con más triángulos (mayor detalle), mientras que el terreno más lejano se renderiza con menos. Esto es crucial para el rendimiento con MDEs grandes.
 - **CesiumJS:** Maneja nativamente el LOD para sus teselas de terreno Quantized Mesh. Solicita y renderiza las teselas apropiadas según la vista.²⁴ El parámetro `maximumScreenSpaceError` controla este comportamiento.

- **Three.js/Babylon.js:** Requiere una implementación personalizada o el uso de bibliotecas/extensiones especializadas en terreno. Las técnicas incluyen la subdivisión del terreno basada en quadrees, geo-mipmapping, o sistemas como la extensión Dynamic Terrain de Babylon.js que soporta LOD.⁷⁰⁵⁹ menciona la carga general de modelos 3D, pero el LOD es una preocupación específica del terreno.
- **Deck.gl:** La TerrainLayer utiliza una TileLayer internamente, lo que implica el manejo de LOD para datos de elevación teselados.⁷⁵
- **LOD de Datos de Línea (si aplica):**
 - Para curvas de nivel vectoriales pre-calculadas u otras geometrías de línea proyectadas, se debe considerar la simplificación de las líneas a niveles de zoom más bajos o el uso de un esquema para cargar datos de línea más detallados solo cuando se está muy cerca.
 - Se aplican estrategias generales para el manejo de grandes conjuntos de datos vectoriales.¹⁰⁵ Si se renderizan una gran cantidad de líneas vectoriales pre-calculadas, estas pueden convertirse en un cuello de botella. Es importante aplicar principios de LOD también a estos datos, como generar versiones simplificadas para diferentes niveles de zoom o filtrar dinámicamente las líneas que son demasiado pequeñas para ser visibles.

6.2. Simplificación y Teselado de Mallas

- **Simplificación de Mallas:** Reducir el número de vértices y caras en una malla preservando su forma general. Esto suele ser parte del proceso de teselado del terreno (por ejemplo, con tin-terrain ³⁶).
- **Teselado:** Dividir el gran modelo de terreno de la Sabana en teselas más pequeñas y manejables que puedan cargarse bajo demanda. Esto es fundamental para el funcionamiento de CesiumJS y la TerrainLayer de Deck.gl.
 - Quantized Mesh ³⁰ es un formato optimizado para la transmisión de mallas de terreno teseladas.

6.3. Técnicas de Optimización de *Shaders*

- **Minimizar Complejidad:** Mantener los *shaders* GLSL lo más simples posible para el efecto deseado. Evitar cálculos innecesarios en el *fragment shader*, que se ejecuta para cada píxel.
- **Búsquedas en Texturas:** Minimizar las búsquedas en texturas si es posible.
- **Bifurcaciones:** Ser consciente de las sentencias condicionales (if) en los *shaders*, ya que a veces pueden afectar el rendimiento en ciertas arquitecturas de GPU, aunque las GPU modernas manejan esto mejor.
- **Precisión:** Usar calificadores de precisión apropiados (highp, mediump, lowp)

para las variables.

6.4. Descarga a Web Workers

Para tareas computacionalmente intensivas en el lado del cliente, como la generación de curvas de nivel mediante Marching Squares a partir de un MDE grande, se debe considerar el uso de Web Workers. Estos permiten realizar dichos cálculos en un hilo separado, evitando que el hilo principal de la interfaz de usuario se congele.

6.5. Almacenamiento en Caché de Datos

Utilizar el almacenamiento en caché del navegador para las teselas de terreno, texturas y otros activos para reducir las descargas redundantes y mejorar los tiempos de carga en visitas posteriores.

6.6. Estructuras de Datos Eficientes

Al manejar grandes cantidades de datos de líneas o puntos en JavaScript, se deben utilizar estructuras de datos y algoritmos eficientes para minimizar el uso de memoria y el tiempo de procesamiento.

Aunque la generación procedural de líneas en tiempo real mediante *shaders* puede ser muy efectiva ³⁵, los *shaders* complejos (por ejemplo, para contornos sugestivos avanzados o líneas de flujo detalladas) que se ejecutan sobre una malla de terreno grande y de alta resolución aún pueden sobrecargar los recursos de la GPU, especialmente en dispositivos de gama baja. La Sabana de Bogotá es un área extensa; incluso con LOD del terreno, la porción visible de la malla puede ser sustancial. Por lo tanto, cualquier desarrollo de *shader* personalizado para estilos de línea debe ir acompañado de pruebas de rendimiento y optimización. Esto podría implicar simplificar la lógica del *shader*, usar menor precisión donde sea aceptable o encontrar atajos algorítmicos.

7. Recomendaciones Estratégicas y Hoja de Ruta de Implementación

Con base en el análisis de los datos, las tecnologías y las técnicas de visualización, esta sección propone una pila tecnológica, un enfoque de desarrollo por fases y aborda los posibles desafíos.

7.1. Pila Tecnológica Propuesta y Justificación

- **Recomendación Principal:** Dada la necesidad de capacidades geoespaciales robustas, manejo eficiente de terreno a gran escala y renderizado de líneas

flexible (incluyendo *shaders* procedurales), se recomienda **CesiumJS** como la biblioteca 3D principal.

- **Justificación:** CesiumJS ofrece un fuerte soporte nativo para terreno (Quantized Mesh ³⁰), un contexto de globo WGS84, excelente LOD y *streaming* ²⁴, y un potente sistema de materiales (Fabric ⁸³) adecuado para efectos de línea basados en *shaders* personalizados como ElevationContour ⁸² y estilos más avanzados.
- **Alternativa/Complementaria:** Si la prioridad es un renderizado no fotorrealista (NPR) extremadamente personalizado y el contexto geoespacial puede simplificarse (por ejemplo, una escena local en lugar de un globo completo), **Three.js** ³¹ ofrece la máxima flexibilidad de *shaders*. Sin embargo, esto requeriría más esfuerzo para la integración de datos geoespaciales y la gestión del terreno.
- **Herramientas de Procesamiento de Datos:**
 - **GDAL:** Para el procesamiento inicial de MDE (mosaico, reproyección, recorte).
 - **tin-terrain** ³⁶ o **Cesium ion** ³⁰: Para convertir GeoTIFF a teselas Quantized Mesh para CesiumJS.
 - Opcionalmente, **QGIS** con complementos como qgis2threejs ⁶² para visualización exploratoria o exportación simple de modelos si se elige la ruta de Three.js para componentes específicos.

7.2. Enfoque de Desarrollo por Fases

Se recomienda un enfoque de desarrollo iterativo y por fases:

- **Fase 1: Visualización Central del Terreno:**
 - Configurar el visor CesiumJS.
 - Adquirir y procesar un MDE adecuado para la Sabana de Bogotá (por ejemplo, IGAC SRTM 30m ¹¹).
 - Convertir el MDE a Quantized Mesh e integrarlo en CesiumJS para su transmisión.
 - Implementar controles básicos de cámara.
- **Fase 2: Implementación de Estilos de Línea Básicos:**
 - Implementar el renderizado de *wireframe* sobre el terreno (por ejemplo, usando el enfoque de material de CesiumJS ⁷⁸).
 - Implementar curvas de nivel procedurales usando el material ElevationContour de CesiumJS. ⁸²
 - Desarrollar controles de interfaz de usuario para alternar estos estilos y ajustar parámetros básicos (por ejemplo, espaciado de curvas).
- **Fase 3: Estilos de Línea Avanzados e Interactividad (Opcional, según requisitos):**

- Desarrollar *shaders* GLSL personalizados para uno o dos estilos de línea avanzados (por ejemplo, líneas de pendiente o jaspeado estilizado) usando Cesium Fabric/CustomShader.
- Implementar la consulta de elevación basada en el ratón.
- Añadir la capacidad de proyectar datos de líneas GeoJSON preexistentes (por ejemplo, ríos).
- **Fase 4: Optimización del Rendimiento y Refinamiento:**
 - Evaluar el rendimiento de la aplicación en diferentes dispositivos.
 - Optimizar *shaders*, configuraciones de LOD y carga de datos.
 - Refinar la UI/UX basándose en pruebas.

Perfeccionar los estilos de línea, especialmente aquellos basados en *shaders* personalizados, es a menudo un proceso iterativo de codificación, pruebas visuales y refinamiento. El plan de desarrollo debe tener en cuenta este ciclo iterativo, especialmente para los estilos de línea avanzados, asignando tiempo para la experimentación y el ajuste de parámetros del *shader* con los datos reales del terreno de la Sabana.

No todos los estilos de línea son igualmente importantes. El enfoque por fases debe priorizar la implementación de aquellos estilos de línea que ofrezcan el mayor valor a los usuarios finales de la visualización de la Sabana, difiriendo los estilos más experimentales o computacionalmente costosos si es necesario. Una consulta con los interesados para entender qué tipos de visualizaciones de líneas son más críticos (por ejemplo, curvas de nivel precisas para planificación, líneas de pendiente para evaluación de riesgos) guiará esta priorización.

7.3. Desafíos Anticipados y Estrategias de Mitigación

- **Desafío: Volumen de Datos y Tiempo de Procesamiento para MDEs Grandes.**
 - **Mitigación:** Comenzar con MDEs de tamaño moderado para el desarrollo. Usar herramientas de teselado eficientes. Considerar el procesamiento basado en la nube (Cesium ion ³⁰) para grandes conjuntos de datos finales.
- **Desafío: Complejidad del Desarrollo de *Shaders* GLSL Personalizados.**
 - **Mitigación:** Aprovechar ejemplos y tutoriales existentes.³⁵ Comenzar con efectos de *shader* más simples e iterar. Asignar tiempo suficiente para el desarrollo y la depuración de *shaders*.
- **Desafío: Lograr un Buen Rendimiento con Múltiples Estilos de Línea Simultáneamente.**
 - **Mitigación:** Evaluar el uso de la GPU. Optimizar los *shaders*. Considerar si todos los estilos de línea necesitan estar activos a la vez o si los usuarios

pueden alternarlos. Usar LOD para los datos de línea cuando sea posible.

- **Desafío: Compatibilidad entre Navegadores y Soporte WebGL.**
 - **Mitigación:** Probar exhaustivamente en los navegadores objetivo. Proporcionar alternativas o mensajes informativos si WebGL o las extensiones requeridas (como OES_standard_derivatives ⁹⁸) no están disponibles.

8. Conclusión

Este informe ha delineado un camino técnico para el desarrollo de una interfaz gráfica web destinada a la visualización 3D con estilos de línea de un modelo de la Sabana de Bogotá.

8.1. Recapitulación de la Viabilidad

El desarrollo de tal interfaz es altamente factible. La disponibilidad de datos geoespaciales pertinentes de fuentes como el IGAC y el SGC ¹, junto con la madurez de las tecnologías web 3D como CesiumJS, Three.js, Babylon.js y Deck.gl ²³, proporciona una base sólida para el proyecto. La clave del éxito radicará en una cuidadosa selección y preparación de datos, una elección informada de la pila tecnológica y una implementación metódica de las técnicas de renderizado y optimización.

8.2. Resumen de Recomendaciones Clave

- Se recomienda **CesiumJS** como la biblioteca 3D principal debido a su robusto manejo de terreno geoespacial y su flexible sistema de materiales.
- Es crucial un **enfoque de desarrollo por fases**, comenzando con la renderización básica del terreno y añadiendo progresivamente estilos de línea y funcionalidades.
- Se debe prestar especial atención a la **evaluación y preparación de los datos MDE**, ya que la calidad de la visualización de líneas depende directamente de ello.
- Las **estrategias de optimización del rendimiento**, especialmente la gestión de LOD tanto para el terreno como para los datos de líneas, son indispensables para asegurar una experiencia de usuario fluida.

8.3. Potencial Futuro y Mejoras

Una visualización 3D con estilos de línea bien implementada de la Sabana de Bogotá no es solo un producto final, sino que puede servir como una potente plataforma fundamental para análisis geoespaciales más avanzados y la toma de decisiones relacionadas con la planificación urbana, el monitoreo ambiental, la evaluación de

riesgos y otras aplicaciones relevantes para la región. La capacidad de integrar capas de datos adicionales (uso del suelo, infraestructura, datos de sensores en tiempo real) sobre este modelo base abre un amplio abanico de posibilidades.

Si bien este informe proporciona una guía técnica experta, el éxito final de la visualización también dependerá de la incorporación de la retroalimentación de sus usuarios previstos a lo largo del proceso de desarrollo. Probar prototipos con usuarios finales y utilizar sus comentarios para refinar los estilos de línea, la interfaz de usuario y la funcionalidad general asegurará que el producto final no solo sea técnicamente sólido, sino también prácticamente útil y valioso para la comunidad a la que sirve.

Obras citadas

1. repositorio.unal.edu.co, fecha de acceso: mayo 14, 2025,
<https://repositorio.unal.edu.co/bitstream/handle/unal/2989/02CAPI01.pdf>
2. Región Capital - Bogotá - Secretaría Distrital de Planeación, fecha de acceso: mayo 14, 2025,
https://www.sdp.gov.co/sites/default/files/documentos/diagnostico_region_capital_dirni.pdf
3. sabanacentrocomovamos.org, fecha de acceso: mayo 14, 2025,
[https://sabanacentrocomovamos.org/municipios/#::~:~:text=Sabana%20Centro%20es%20una%20de,Zipaquir%C3%A1%2C%20Cogua%20y%20Nemoc%C3%B3n\).](https://sabanacentrocomovamos.org/municipios/#::~:~:text=Sabana%20Centro%20es%20una%20de,Zipaquir%C3%A1%2C%20Cogua%20y%20Nemoc%C3%B3n).)
4. app.ingemmet.gob.pe, fecha de acceso: mayo 14, 2025,
<https://app.ingemmet.gob.pe/biblioteca/pdf/CPG14-034.pdf>
5. definicion.de, fecha de acceso: mayo 14, 2025,
<https://definicion.de/sabana/#::~:~:text=En%20Colombia%20existe%20la%20saban a,de%20la%20ropa%20de%20cama.>
6. dialnet.unirioja.es, fecha de acceso: mayo 14, 2025,
<https://dialnet.unirioja.es/descarga/articulo/4902557.pdf>
7. Minambiente pone en consulta pública lineamientos para ..., fecha de acceso: mayo 14, 2025,
<https://www.minambiente.gov.co/minambiente-pone-en-consulta-publica-lineamientos-para-ordenamiento-ambiental-de-la-sabana-de-bogota/>
8. repositorio.unal.edu.co, fecha de acceso: mayo 14, 2025,
<https://repositorio.unal.edu.co/bitstream/handle/unal/51340/1026553825.2014.pdf?sequence=3&isAllowed=y>
9. (PDF) Bachelor Thesis - Aproximación a la cuantificación de pasivos ambientales, fecha de acceso: mayo 14, 2025,
https://www.researchgate.net/publication/281832243_Bachelor_Thesis_-_Aproximacion_a_la_cuantificacion_de_pasivos_ambientales
10. www.car.gov.co, fecha de acceso: mayo 14, 2025,
<https://www.car.gov.co/uploads/files/60d35674bb455.pdf>
11. Modelo Digital de Elevación. Colombia. SRTM 30 Metros. Año 2011, fecha de acceso: mayo 14, 2025,

- <http://www.colombiaenmapas.gov.co/?u=0&t=23&servicio=159>
12. Cartografía básica - Colombia en mapas, fecha de acceso: mayo 14, 2025, <https://www.colombiaenmapas.gov.co/?u=0&t=23>
 13. Instituto Geográfico Agustín Codazzi: Bienvenido, fecha de acceso: mayo 14, 2025, <https://www.igac.gov.co/>
 14. Datos para el OT - Colombia OT, fecha de acceso: mayo 14, 2025, <https://www.colombiaot.gov.co/datosot/>
 15. Curva de Nivel - Bogotá - Ideca, fecha de acceso: mayo 14, 2025, <https://www.ideca.gov.co/recursos/mapas/curva-de-nivel>
 16. 369 4000 Ext. 91331 Bogotá www.igac.gov.co, fecha de acceso: mayo 14, 2025, https://www.igac.gov.co/sites/default/files/transparencia/planeacionypresupuesto/informe_de_gestion_cierre_2019_1.pdf
 17. Acceso, visualización y consulta de la información georreferenciada ..., fecha de acceso: mayo 14, 2025, <https://www2.sgc.gov.co/sgc/mapas/paginas/geoportal.aspx>
 18. MIIG-Resultado Búsqueda - SGC, fecha de acceso: mayo 14, 2025, <https://miig.sgc.gov.co/Paginas/Resultados.aspx?k=ZONIFICACION%20N+GEOLOGIA%20DE+LA+SABANA+DE+BOGOTA>
 19. Búsqueda de Información Geocientífica - MIIG, fecha de acceso: mayo 14, 2025, <https://miig.sgc.gov.co/Paginas/Resultados.aspx?k=geologia+plancha+227>
 20. MIIG-Resultado Búsqueda - Servicio Geológico Colombiano, fecha de acceso: mayo 14, 2025, <https://miig.sgc.gov.co/Paginas/Resultados.aspx?k=geologia+de+la+sabana+de+bogota>
 21. Modelo Digital de Superficie Zona Urbana de Bogotá - Conjunto de datos, fecha de acceso: mayo 14, 2025, <https://datosabiertos.bogota.gov.co/dataset/modelo-digital-de-superficie-zona-urbana-de-bogota>
 22. Modelo Digital de Superficie Zona Urbana de Bogotá - Ideca, fecha de acceso: mayo 14, 2025, <https://www.ideca.gov.co/recursos/mapas/modelo-digital-de-superficie-zona-urbana-de-bogota>
 23. 3D Maps for JavaScript API now in Preview: start building your own immersive maps experience today - Google Maps Platform, fecha de acceso: mayo 14, 2025, <https://mapsplatform.google.com/resources/blog/3d-maps-for-javascript-api-now-in-preview-start-building-your-own-immersive-maps-experience-today/>
 24. Cesium terrain and OSM buildings lod discrepancies, fecha de acceso: mayo 14, 2025, <https://community.cesium.com/t/cesium-terrain-and-osm-buildings-lod-discrepancies/37913>
 25. Adaptive Contouring in Fragment Shaders / Ricky Reusser - Observable, fecha de acceso: mayo 14, 2025, <https://observablehq.com/@rreusser/locally-scaled-domain-coloring-part-1-contour-plots>
 26. Colombia en mapas, fecha de acceso: mayo 14, 2025,

- <https://www.colombiaenmapas.gov.co/>
27. ANEXOS CAPÍTULO 4.1 1 ANEXO 4.1-1. ESTUDIO HIDROGEOLÓGICO DETALLADO DE BOGOTÁ, CON ÉNFASIS EN LA ZONA NORTE DE LA CIUDAD (ZO - CAR, fecha de acceso: mayo 14, 2025, <https://www.car.gov.co/uploads/files/5c59f8cfe9580.pdf>
 28. INFORME DEL RECORRIDO POR EL RÍO BOGOTÁ, PARA LA IDENTIFICACIÓN DE PUNTOS CRÍTICOS EN SU DINÁMICA HIDRÁULICA 1. INTRODUCI - SIRE, fecha de acceso: mayo 14, 2025, <http://logina.sire.gov.co/riobogota/documentos/docs/infoRecorridoAcuaticoAereo.pdf>
 29. Río Bogotá - Wikipedia, la enciclopedia libre, fecha de acceso: mayo 14, 2025, https://es.wikipedia.org/wiki/R%C3%ADo_Bogot%C3%A1
 30. Terrain - Cesium, fecha de acceso: mayo 14, 2025, <https://cesium.com/learn/3d-tiling/ion-tile-terrain/>
 31. Creating 3D Terrain Maps from GeoTIFF Files with Three.js - Spatial Dev Guru, fecha de acceso: mayo 14, 2025, <https://spatial-dev.guru/2024/11/30/creating-3d-terrain-maps-from-geotiff-files-with-three-js/>
 32. Create 3d surface from geotiffs - FME Community - Safe Software, fecha de acceso: mayo 14, 2025, <https://community.safe.com/data-7/create-3d-surface-from-geotiffs-17351>
 33. SvenPfiffner/GeoTiffConverter: This Python package and web UI allow you to seamlessly convert GeoTiff data into 3D meshes, making geospatial data visualization and analysis a breeze. Whether you're working with elevation maps, terrain models, or any other geospatial information stored in GeoTiff files, this tool simplifies the process of turning raw data - GitHub, fecha de acceso: mayo 14, 2025, <https://github.com/SvenPfiffner/GeoTiffConverter>
 34. GeoTIFF - Blue Marble Geographics, fecha de acceso: mayo 14, 2025, <https://www.blumablegeo.com/knowledgebase/global-mapper-26-1/Formats/GeoTIFF.htm>
 35. 9 3D Terrain Visualization Methods That Transform Digital Landscapes - Map Library, fecha de acceso: mayo 14, 2025, <https://www.maplibrary.org/1520/3d-terrain-visualization-methods/>
 36. heremaps/tin-terrain: A command-line tool for converting ... - GitHub, fecha de acceso: mayo 14, 2025, <https://github.com/heremaps/tin-terrain>
 37. Heightmaps — Terrain3D 1.1.0 documentation, fecha de acceso: mayo 14, 2025, <https://terrain3d.readthedocs.io/en/latest/docs/heightmaps.html>
 38. fecha de acceso: diciembre 31, 1969, <https://adminmiignew.sgc.gov.co/Lists/RecursosSGC/Metadatos.aspx?ID=16155>
 39. fecha de acceso: diciembre 31, 1969, <https://adminmiignew.sgc.gov.co/Lists/RecursosSGC/Metadatos.aspx?ID=20>
 40. Generate 3D tiles from a DEM : r/QGIS - Reddit, fecha de acceso: mayo 14, 2025, https://www.reddit.com/r/QGIS/comments/1jsz9nu/generate_3d_tiles_from_a_dem/
 41. Quantized Mesh Serving / SceneMode - Cesium Community, fecha de acceso:

58. 22b How to color terrain from heightmap three.js - YouTube, fecha de acceso: mayo 14, 2025, <https://www.youtube.com/watch?v=G0hWjD0n46c>
59. How to Create Dynamic 3D Environments Using WebGL - PixelFreeStudio Blog, fecha de acceso: mayo 14, 2025, <https://blog.pixelfreestudio.com/how-to-create-dynamic-3d-environments-using-webgl/>
60. erichlof/THREE.js-PathTracing-Renderer: Real-time PathTracing with global illumination and progressive rendering, all on top of the Three.js WebGL framework. Click here for Live Demo: https://erichlof.github.io/THREE.js-PathTracing-Renderer/Geometry_Showcase.html - GitHub, fecha de acceso: mayo 14, 2025, <https://github.com/erichlof/THREE.js-PathTracing-Renderer>
61. Add a 3D model using three.js - MapLibre GL JS, fecha de acceso: mayo 14, 2025, <https://maplibre.org/maplibre-gl-js/docs/examples/add-3d-model/>
62. 3D viz with QGIS & three.js | Free and Open Source GIS Ramblings, fecha de acceso: mayo 14, 2025, <https://anitagraser.com/2014/03/15/3d-viz-with-qgis-three-js/>
63. QC20/Endless-Mountains: This repository contains a WebGL-based terrain visualization using Three.js, a popular JavaScript library for creating 3D graphics in the browser. The project dynamically generates a terrain landscape with mountains and valleys that evolve over time. - GitHub, fecha de acceso: mayo 14, 2025, <https://github.com/QC20/Endless-Mountains/>
64. Babylon.js: Powerful, Beautiful, Simple, Open - Web-Based 3D At Its Best, fecha de acceso: mayo 14, 2025, <https://www.babylonjs.com/>
65. 3D Gizmos - Babylon.js Playground, fecha de acceso: mayo 14, 2025, <https://www.babylonjs-playground.com/#0LMZ2Y#6>
66. Babylon 3D Geography - Demos and projects, fecha de acceso: mayo 14, 2025, <https://forum.babylonjs.com/t/babylon-3d-geography/51352>
67. Babylon.js Digital Twins and IoT, fecha de acceso: mayo 14, 2025, <https://www.babylonjs.com/digitalTwinIot/>
68. Getting Started - Chapter 8 - Have a Look Around - Babylon.js Documentation, fecha de acceso: mayo 14, 2025, <https://doc.babylonjs.com/features/introductionToFeatures/chap8/camera/>
69. ArcRotateCamera in Babylon.js - Catherine Thomsen, fecha de acceso: mayo 14, 2025, <https://catherinethomsen.com/arcrotatecamera-in-babylon-js/>
70. Dynamic Terrain - Babylon.js Documentation, fecha de acceso: mayo 14, 2025, <https://doc.babylonjs.com/communityExtensions/dynamicTerrains/>
71. Dynamic Terrain - Babylon.js Documentation, fecha de acceso: mayo 14, 2025, <https://doc.babylonjs.com/communityExtensions/dynamicTerrains>
72. Ideas for rendering and optimizing complex scenes - Babylon JS Forum, fecha de acceso: mayo 14, 2025, <https://forum.babylonjs.com/t/ideas-for-rendering-and-optimizing-complex-scenes/47511>
73. Contour lines on a closed mesh? - Babylon JS Forum, fecha de acceso: mayo 14, 2025, <https://forum.babylonjs.com/t/contour-lines-on-a-closed-mesh/42669>

74. Marching squares - Questions - Babylon.js, fecha de acceso: mayo 14, 2025, <https://forum.babylonjs.com/t/marching-squares/13722>
75. TerrainLayer | deck.gl, fecha de acceso: mayo 14, 2025, <https://deck.gl/docs/api-reference/geo-layers/terrain-layer>
76. Layer Class - Deck.gl, fecha de acceso: mayo 14, 2025, <https://deck.gl/docs/api-reference/core/layer>
77. ContourLayer | deck.gl, fecha de acceso: mayo 14, 2025, <https://deck.gl/docs/api-reference/aggregation-layers/contour-layer>
78. Cesium3DTileset - Cesium Documentation, fecha de acceso: mayo 14, 2025, <https://cesium.com/downloads/cesiumjs/releases/1.43/Build/Documentation/Cesium3DTileset.html>
79. three.js docs, fecha de acceso: mayo 14, 2025, <https://threejs.org/docs/#api/en/materials/MeshStandardMaterial>
80. Introduction To Materials | Babylon.js Documentation, fecha de acceso: mayo 14, 2025, https://doc.babylonjs.com/features/featuresDeepDive/materials/using/materials_introduction
81. Babylon.js Wireframe - Tutorialspoint, fecha de acceso: mayo 14, 2025, https://www.tutorialspoint.com/babylonjs/babylonjs_wireframe.htm
82. Working with Cesium World Bathymetry in CesiumJS, fecha de acceso: mayo 14, 2025, <https://cesium.com/blog/2024/01/29/cesium-world-bathymetry-in-cesiumjs/>
83. Material - Cesium Documentation, fecha de acceso: mayo 14, 2025, <https://cesium.com/learn/ion-sdk/ref-doc/Material.html>
84. CustomShader - Cesium Documentation, fecha de acceso: mayo 14, 2025, <http://cesium.xin/cesium/en/Documentation1.95/CustomShader.html>
85. Flow Lines Material or Appearance - CesiumJS - Cesium Community, fecha de acceso: mayo 14, 2025, <https://community.cesium.com/t/flow-lines-material-or-appearance/5260>
86. How to use cesium.js to make a line with flowing texture like a picture - Stack Overflow, fecha de acceso: mayo 14, 2025, <https://stackoverflow.com/questions/54819023/how-to-use-cesium-js-to-make-a-line-with-flowing-texture-like-a-picture>
87. Combining multiple ElevationContour materials - CesiumJS - Cesium Community, fecha de acceso: mayo 14, 2025, <https://community.cesium.com/t/combining-multiple-elevationcontour-materials/14337>
88. Fabric · CesiumGS/cesium Wiki - GitHub, fecha de acceso: mayo 14, 2025, <https://github.com/CesiumGS/cesium/wiki/Fabric>
89. Wireframe / Texture hybrid option - CesiumJS - Cesium Community, fecha de acceso: mayo 14, 2025, <https://community.cesium.com/t/wireframe-texture-hybrid-option/1868>
90. OrbitControls - three.js docs, fecha de acceso: mayo 14, 2025, <https://threejs.org/docs/#examples/ko/controls/OrbitControls>
91. Orbit controls - Three.js, fecha de acceso: mayo 14, 2025,

- <https://threejs.org/docs/examples/en/controls/OrbitControls.html>
92. ScreenSpaceCameraController - Cesium Documentation, fecha de acceso: mayo 14, 2025,
<https://help.agi.com/STKWebVisualizationLibrary/ScreenSpaceCameraController.html>
 93. Camera - Cesium Documentation, fecha de acceso: mayo 14, 2025,
<https://cesium.com/downloads/cesiumjs/releases/b30/Build/Documentation/Camera.html>
 94. Cesium Community - Cesium Community Forum, fecha de acceso: mayo 14, 2025, <https://community.cesium.com/>
 95. three.js forum, fecha de acceso: mayo 14, 2025, <https://discourse.threejs.org/>
 96. How to add contours to cesium tiles? - CesiumJS, fecha de acceso: mayo 14, 2025,
<https://community.cesium.com/t/how-to-add-contours-to-cesium-tiles/6688>
 97. DEM Contour Lines - Sentinel Hub custom scripts, fecha de acceso: mayo 14, 2025, <https://custom-scripts.sentinel-hub.com/custom-scripts/dem/contour-lines/>
 98. Create topography/contour lines with GLSL - WebGL - Stack Overflow, fecha de acceso: mayo 14, 2025,
<https://stackoverflow.com/questions/34297337/create-topography-contour-lines-with-glsl-webgl>
 99. TristanAntonsen/marching-squares - GitHub, fecha de acceso: mayo 14, 2025,
<https://github.com/TristanAntonsen/marching-squares>
 100. RaumZeit/MarchingSquares.js: A JavaScript implementation of the Marching Squares algorithm featuring IsoContour and IsoBand computation - GitHub, fecha de acceso: mayo 14, 2025,
<https://github.com/RaumZeit/MarchingSquares.js/>
 101. d3/d3-contour: Compute contour polygons using marching squares. - GitHub, fecha de acceso: mayo 14, 2025, <https://github.com/d3/d3-contour>
 102. Marching squares - Wikipedia, fecha de acceso: mayo 14, 2025,
https://en.wikipedia.org/wiki/Marching_squares
 103. marching squares - Youpi !, fecha de acceso: mayo 14, 2025,
<http://barradeau.com/blog/?p=391>
 104. fecha de acceso: diciembre 31, 1969,
<https://github.com/RaumZeit/MarchingSquares.js>
 105. Tips and tricks on how to handle large scale drawings in CAD - PVcase Help Center, fecha de acceso: mayo 14, 2025,
<https://help.pvcase.com/hc/en-us/articles/35595036136211-Tips-and-tricks-on-how-to-handle-large-scale-drawings-in-CAD>
 106. 3D Terrain Modeling with CAD Software - Using Plan7Architect, fecha de acceso: mayo 14, 2025,
<https://plan7architect.com/3d-terrain-modeling-with-cad-software-using-plan7architect-ai/>
 107. OpenLayers display labels on contour lines using WebGL - Stack Overflow, fecha de acceso: mayo 14, 2025,
<https://stackoverflow.com/questions/78847874/openlayers-display-labels-on-con>

[tour-lines-using-webgl](#)

108. Illustrating Terrains using Direction of Slope and Lighting - Commission on Mountain Cartography, fecha de acceso: mayo 14, 2025, https://mountaincartography.icaci.org/publications/papers/papers_nuria_04/buchin.pdf
109. A flowing WebGL gradient, deconstructed - Alex Harri, fecha de acceso: mayo 14, 2025, <https://alexharri.com/blog/webgl-gradients>
110. Basic WebGL Meshes - IMAGINE, fecha de acceso: mayo 14, 2025, <https://imagine.inrialpes.fr/people/Francois.Faure/htmlCourses/WebGL/IntroMeshes.html>
111. Line drawings via abstracted shading - POSTECH Computer Graphics Laboratory, fecha de acceso: mayo 14, 2025, <https://cg.postech.ac.kr/papers/line-drawing-s07.pdf>
112. Generating contour in the mesh - Questions - three.js forum, fecha de acceso: mayo 14, 2025, <https://discourse.threejs.org/t/generating-contour-in-the-mesh/42288>
113. I wanna render a 3d model with lines - Questions - three.js forum, fecha de acceso: mayo 14, 2025, <https://discourse.threejs.org/t/i-wanna-render-a-3d-model-with-lines/23415>
114. Visualizing Imagery - Cesium, fecha de acceso: mayo 14, 2025, <https://cesium.com/learn/cesiumjs-learn/cesiumjs-imagery/>
115. Maya Help | Create and visualize a GLSL shader | Autodesk, fecha de acceso: mayo 14, 2025, <https://help.autodesk.com/view/MAYAUL/2025/ENU/?guid=GUID-41D74556-0699-4F4E-8382-F9E69FBFC158>